Naming Conventions

Newcomers to Rails are sometimes puzzled by the way it automatically handles the naming of things. They're surprised that they call a model class `Person` and Rails somehow knows to go looking for a database table called `people`. In this section, you'll learn how this implicit naming works.

The rules here are the default conventions used by Rails. You can override all of these conventions using configuration options.

Mixed Case, Underscores, and Plurals

We often name variables and classes using short phrases. In Ruby, the convention is to have variable names where the letters are all lowercase and words are separated by underscores. Classes and modules are named differently: there are no underscores, and each word in the phrase (including the first) is capitalized. (We'll call this *mixed case*, for fairly obvious reasons.) These conventions lead to variable names such as `order_status` and class names such as `LineItem`.

Rails takes this convention and extends it in two ways. First, it assumes that database table names, such as variable names, have lowercase letters and underscores between the words. Rails also assumes that table names are always plural. This leads to table names such as `orders` and `third_parties`.

On another axis, Rails assumes that files are named using lowercase with underscores.

Rails uses this knowledge of naming conventions to convert names automatically. For example, your application might contain a model class that handles line items. You'd define the class using the Ruby naming convention, calling it `LineItem`. From this name, Rails would automatically deduce the following:

- The corresponding database table will be called `line_items`. That's the class